7 slab 内存池的设计与实现 2
7.1 伙伴系统与 Slab 内存池的关系 2
7.2 slab 对象池在内核中的应用场景 3
7.3 slab、slub、slob 三者的关系 6
7.4 slab 对象的内存布局 6
7.5 slab 的总体架构设计 11
7.5.1 slab 的基础信息管理 12
7.5.2 slab 的组织架构 16
7.6 slab 内存分配原理 21
7.6.1 从本地 cpu 缓存中直接分配 21
7.6.3 从本地 cpu 缓存 partial 列表中分配 22
7.6.4 从 NUMA 节点缓存中分配 24
7.6.5 从伙伴系统中重新申请 slab 27
7.7 slab 内存释放原理 29
7.7.1 释放对象所属 slab 在 cpu 本地缓存中 29
7.7.2 释放对象所属 slab 在 cpu 本地缓存 partial 列表中 31
7.7.3 释放对象所属 slab 从 full slab 变为了 partial slab 32
7.7.4 释放对象所属 slab 从 full slab 变为了 partial slab 33

7 slab 内存池的设计与实现

7.1 伙伴系统与 Slab 内存池的关系

从伙伴系统核心数据结构，以及伙伴系统内存分配原理的相关内容来看，伙伴系统管理物理内存的最小单位是物理内存页 page。也就是说，向伙伴系统申请内存时，至少要申请一个物理内存页。

从内核实际运行过程中来看，无论是从内核态还是从用户态的角度来说，对于内存的需求量往往是以字节为单位，通常是几十字节到几百字节不等，远远小于一个页面的大小。如果我们仅仅为了这几十字节的内存需求，而专门为其分配一整个内存页面，是对宝贵内存资源的一种巨大浪费。

为了避免不必要的内存浪费。内核中设计了 slab 内存池专门应对小内存频繁的分配和释放的场景。slab 首先会向伙伴系统一次性申请一个或者多个物理内存页面，正是这些物理内存页组成了 slab 内存池。随后 slab 内存池会将这些连续的物理内存页面划分成多个大小相同的小内存块出来，同一种 slab 内存池下，划分出来的小内存块尺寸是一样的。内核会针对不同尺寸的小内存分配需求，预先创建出多个 slab 内存池出来。

这种小内存在内核中的使用场景非常频繁，比如，内核中那些经常使用，需要频繁申请释放的一些核心数据结构对象：task_struct 对象，mm_struct 对象，struct page 对象，struct file 对象，socket 对象等。而创建这些内核核心数据结构对象以及为这些核心对象分配内存，销毁这些内核对象以及释放相关的内存是需要性能开销的。

在内核中分别为每一个需要被内核频繁创建和释放的核心对象创建一个专属的 slab 对象池，这些内核对象专属的 slab 对象池会根据其所管理的具体内核对象所占用内存的大小 size，将一个或者多个完整的物理内存页按照这个 size 划分出多个大小相同的小内存块，每个小内存块用于存储预先创建好的内核对象。

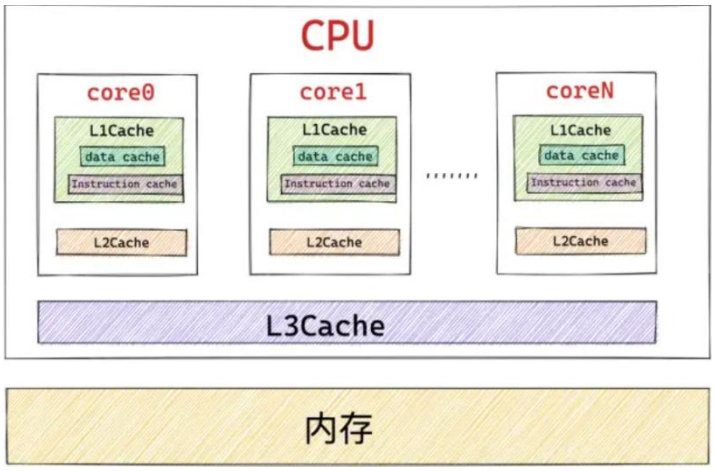
这样一来，当内核需要频繁分配和释放内核对象时，就可以直接从相应的 slab 对象池中申请和释放内核对象，避免了链路比较长的内存分配与释放过程，极大地提升了性能。这是一种池化思想的应用。

将内核中的核心数据结构对象，池化在 slab 对象池中，除了可以避免内核对象频繁反复初始化和相关内存分配，频繁反复销毁对象和相关内存释放的性能开销之外，其实还有很多好处，比如：

1. 利用 CPU 高速缓存提高访问速度。当一个对象被直接释放回 slab 对象池中的时候，这个内核对象还是“热的”，仍然会驻留在 CPU 高速缓存中。如果这时，内核继续向 slab 对象池申请对象，slab 对象池会优先把这个刚刚释放“热的”对象分配给内核使用，因为对象很大概率仍然驻留在 CPU 高速缓存中，所以内核访问起来速度会更快。

2. 伙伴系统只能分配 2 的次幂个完整的物理内存页，这会引起占用高速缓存以及 TLB 的空间较大，导致一些不重要的数据驻留在 CPU 高速缓存中占用宝贵的缓存空间，而重要的数据却被置换到内存中。 slab 对象池针对小内存分配场景，可以有效的避免这一点。

3. 调用伙伴系统的操作会对 CPU 高速缓存 L1Cache 中的 Instruction Cache（指令高速缓存）和 Data Cache （数据高速缓存）有污染，因为对伙伴系统的长链路调用，相关的一些指令和数据必然会填充到 Instruction Cache 和 Data Cache 中，从而将频繁使用的一些指令和数据挤压出去，造成缓存污染。而在内核空间中越浪费这些缓存资源，那么在用户空间中的进程就会越少的得到这些缓存资源，造成性能的下降。 slab 对象池极大的减少了对伙伴系统的调用，防止了不必要的 L1Cache 污染。



4. 使用 slab 对象池可以充分利用 CPU 高速缓存，避免多个对象对同一 cache line 的争用。如果对象直接存储排列在伙伴系统提供的内存页中的话（不受 slab 管理），那么位于不同内存页中具有相同偏移的对象很可能会被放入同一个 cache line 中，即使其他 cache line 还是空的。

7.2 slab 对象池在内核中的应用场景

1. 使用 fork() 系统调用创建进程的时候，内核需要使用 task_struct 专属的 slab 对象池分配 task_struct 对象。

```
static struct task_struct *dup_task_struct(struct task_struct *orig, int
node)
{
                ......
    struct task_struct *tsk;
                ......
    // 从 task_struct 对象专属的 slab 对象池中申请 task_struct 对象
    tsk = alloc_task_struct_node(node);
                ......
}
```

2. 为进程创建虚拟内存空间的时候，内核需要使用 mm_struct 专属的 slab 对象池分配 mm_struct 对象。

```
static struct mm_struct *dup_mm(struct task_struct *tsk,
                struct mm_struct *oldmm)
{
     ......
    struct mm_struct *mm;
    ......
    // 从 mm_struct 对象专属的 slab 对象池中申请 mm_struct 对象
    mm = allocate_mm();
    ......
    }
```

3. 向页高速缓存 page cache 查找对应的文件缓存页时，内核需要使用 struct page 专属的 slab 对象池分配 struct page 对象。

```
struct folio *__filemap_get_folio(struct address_space *mapping, pgoff_t
index,
        fgf_t fgp_flags, gfp_t gfp)
{
    struct folio *folio;

repeat:
    //在 radix_tree（page cache）中根据缓存页 offset 查找缓存页
    folio = filemap_get_entry(mapping, index);
    if (xa_is_value(folio))
        folio = NULL;
   // 缓存页不存在的话，跳转到 no_page 处理逻辑
    if (!folio)
        goto no_page;
    ......

no_page:
      ......
      // 从 page 对象专属的 slab 对象池中申请 page 对象
      folio = filemap_alloc_folio(alloc_gfp, order);
     .......
    err = filemap_add_folio(mapping, folio, index, gfp);
    ......

    return folio;
}
```

4. 使用 open 系统调用打开一个文件时，内核需要使用 struct file 专属的 slab 对象池分配 struct file 对象。

```
struct file *do_filp_open(int dfd, struct filename *pathname,
        const struct open_flags *op)
{
    ......
    struct file *filp;
    ......

    // 分配 struct file 内核对象
    filp = path_openat(&nd, op, flags | LOOKUP_REVAL);
    ......

    return filp;
}
```

```
static struct file *path_openat(struct nameidata *nd,
            const struct open_flags *op, unsigned flags)
{
    struct file *file;
    ......
    // 从 struct file 对象专属的 slab 对象池中申请 struct file 对象
    file = alloc_empty_file(op->open_flag, current_cred());
    ......

}
```

5. 服务端网络应用程序使用 accpet 系统调用接收客户端的连接时，内核需要使用 struct socket 专属的 slab 对象池为新进来的客户端连接分配 socket 对象。

```
SYSCALL_DEFINE4(accept4, int, fd, struct sockaddr __user *, upeer_sockaddr,
        int __user *, upeer_addrlen, int, flags)
{
    return __sys_accept4(fd, upeer_sockaddr, upeer_addrlen, flags);
}
```

```
int __sys_accept4(int fd, struct sockaddr __user *upeer_sockaddr,
          int __user *upeer_addrlen, int flags)
{
       ......
        ret = __sys_accept4_file(f.file, upeer_sockaddr,
                     upeer_addrlen, flags);
       ......
    return ret;
}
```

```
static int __sys_accept4_file(struct file *file, struct sockaddr __user
*upeer_sockaddr,
                 int __user *upeer_addrlen, int flags)
{
      ......
    newfile = do_accept(file, 0, upeer_sockaddr, upeer_addrlen,
                flags);
   ......
    return newfd;
}
```

```
struct file *do_accept(struct file *file, unsigned file_flags,
```

```
                struct sockaddr __user *upeer_sockaddr,
                int __user *upeer_addrlen, int flags)
{
      struct socket *sock, *newsock;

      ......
      //从 struct socket 对象专属的 slab 对象池中申请 struct socket 对象
      newsock = sock_alloc();
      ......

}
```

等其它专属的 slab 对象池。

7.3 slab、slub、slob 三者的关系

slab 最早是由 Sun 公司的 Jeff Bonwick 在 Solaris 2.4 系统中设计并实现的，由于 Jeff Bonwick 公开了 slab 的实现方法，因此被 Linux 所借鉴并于 1996 年在 Linux 2.0 版本中引入了 slab，用于 Linux 内核早期的小内存分配场景。

由于 slab 的实现非常复杂，slab 中拥有多种存储对象的队列，队列管理开销比较大，slab 元数据比较臃肿，对 NUMA 架构的支持臃肿繁杂（slab 引入时内核还没支持 NUMA），这样导致 slab 内部为了维护这些自身元数据管理结构就得花费大量的内存空间，这在配置有超大容量内存的服务器上，内存的浪费是非常可观的。

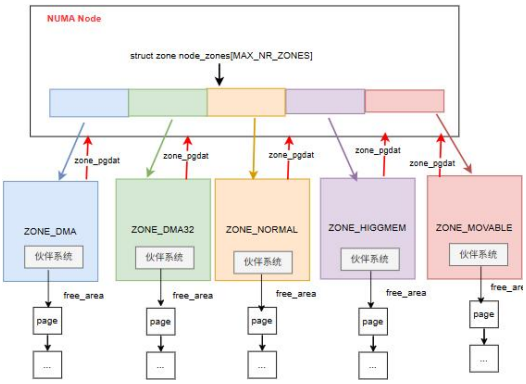
针对 slab 的不足，Christoph Lameter 在 2.6.22 版本（2007 年发布）中引入了新的 slub 实现。slub 简化了 slab 一些复杂的设计，同时保留了 slab 的基本思想，摒弃了 slab 众多管理队列的概念，并针对多处理器，NUMA 架构进行优化，放弃了效果不太明显的 slab 着色机制。slub 与 slab 相比，提高了性能，吞吐量，并降低了内存的浪费。成为现在内核中常用的 slab 实现。

slob 是在内核 2.6.16 版本（2006 年发布）引入的，它是专门为嵌入式小型机器小内存的场景设计的，所以实现上很精简，能在小型机器上提供很不错的性能。而内核中关于内存池（小内存分配器）的相关 API 接口函数均是以 slab 命名的.

可以通过配置的方式来平滑切换以上三种 slab 的实现。slub 对象池被大规模运用在服务器 Linux 操作系统。

7.4 slab 对象的内存布局

内核会把那些频繁使用的核心对象统一放在 slab 对象池中管理，每一个核心对象对应一个专属的 slab 对象池，以便提升核心对象的分配，访问，释放相关操作的性能。



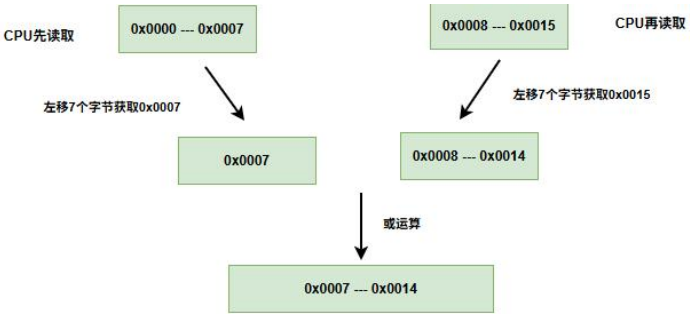
如上图所示，slab 对象池在内存管理系统中的架构层次是基于伙伴系统之上构建的，slab 对象池会一次性向伙伴系统申请一个或者多个完整的物理内存页，在这些完整的内存页内在逐步划分出一小块一小块的内存块出来，而这些小内存块的尺寸就是 slab 对象池所管理的内核核心对象占用的内存大小。

以一个最简单的物理内存页 page 开始，一步一步的推演 slab 的整个架构设计与实现。设计一个对象池，首先最直观最简单的办法就是先向伙伴系统申请一个内存页，然后按照需要被池化对象的尺寸 object size，把内存页划分为一个一个的内存块，每个内存块尺寸就是 object size，如下图所示。事实上，slab 对象池可以根据情况向伙伴系统一次性申请多个内存页。此处，以一个内存页为例。

在实际应用中，对象的 object size 可以是任意的，并不是内存对齐的，CPU 访问一块没有进行对齐的内存比访问对齐的内存速度要慢一倍。

因为 CPU 向内存读取数据的单位是根据 word size 来的，在 64 位处理器中 word size = 8 字节，所以 CPU 向内存读写数据的单位为 8 字节。CPU 只能一次性向内存访问按照 word size (8 字节) 对齐的内存地址，如果 CPU 访问一个未进行 word size 对齐的内存地址，就会经历两次访存操作。

比如，需要访问 0x0007 - 0x0014 这样一段没有对 word size 进行对齐的内存，CPU 只能先从 0x0000 - 0x0007 读取 8 个字节出来先放入结果寄存器中并左移 7 个字节（目的是只获取 0x0007 ），然后 CPU 在从 0x0008 - 0x0015 读取 8 个字节出来放入临时寄存器中并右移 1 个字节（目的是获取 0x0008 - 0x0014 ）最后与结果寄存器或运算。最终得到 0x0007 - 0x0014 地址段上的 8 个字节。



可以看出，CPU 访问一段未进行 word size 对齐的内存，需要两次访存操作。内存对齐的

好处还有很多，比如，CPU 访问对齐的内存都是原子性的，对齐内存中的数据会独占 cache line ，不会与其他数据共享 cache line，避免 false sharing。

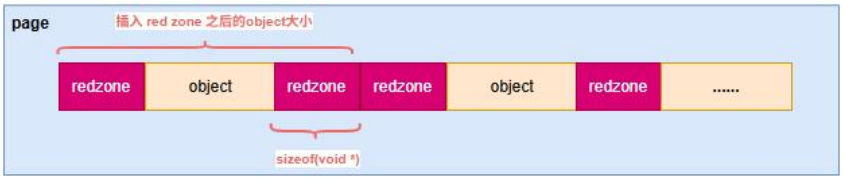
所以不能简单的按照对象尺寸 object size 来划分内存块，而是需要考虑到对象内存地址要按照 word size 进行对齐。于是上面的 slab 对象池的内存布局又有了新的变化。

如果被池化对象的尺寸 object size 本来就是和 word size 对齐的，就不需要做任何事情，但是如果 object size 没有和 word size 对齐，就需要填充一些字节，目的是要让对象的 object size 按照 word size 进行对齐，提高 CPU 访问对象的速度。

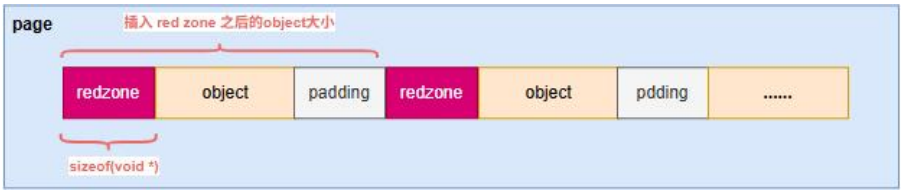
另外，内核为了应对内存读写越界的场景，于是在对象内存的周围插入了一段不可访问的内存区域，这些内存区域用特定的字节 0xbb 填充，当进程访问的到内存是 0xbb 时，表示已经越界访问了。这段内存区域在 slab 中的术语为 red zone，大家可以理解为红色警戒区域。

对于 red zone 区域的插入处理存在两种情况：

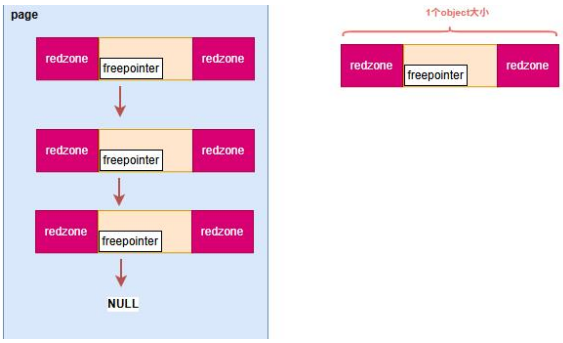
1. 如果对象尺寸 object size 本身就是 word size 对齐的，那么就需要在对象左右两侧填充两段 red zone 区域，red zone 区域的长度一般就是 word size 大小。



2. 如果对象尺寸 object size 是通过填充 padding 之后，才与 word size 对齐。内核会巧妙的利用对象右边的这段 padding 填充区域作为 red zone。只需要额外的在对象内存区域的左侧填充一段 red zone 即可。

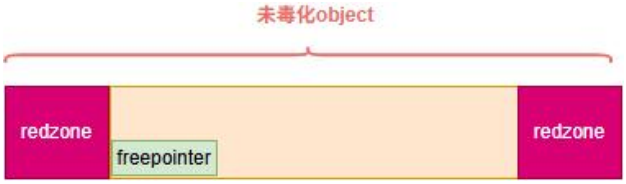


对象在 slab 中没有被分配出去使用的时候，用户并不关心对象所占的内存中具体存放的内容。于是，内核把指向下一个空闲对象的 freepointer 指针直接存放在对象所占内存（object size）中，这样避免了为 freepointer 指针单独再分配内存空间。巧妙的利用了对象所在的内存空间（object size）。利用 freepointer 指针将内存中的空闲对象串起来起来。



另外，内核会在对象所占的内存空间中填充一些特殊的字符用来表示对象的不同状态。当 slab 刚刚从伙伴系统中申请出来，并初始化划分物理内存页中的对象内存空间时，内核会将对象的 object size 内存区域用特殊字节 0x6b 填充，并用 0xa5 填充对象 object size 内存区域的最后一个字节表示填充完毕。或者当对象被释放回 slab 对象池中的时候，也会用这些字节填充对象的内存区域。

这种通过在对象内存区域填充特定字节表示对象的特殊状态的行为，在 slab 中有一个专门的术语叫做 SLAB_POISON （SLAB 中毒）。但是，是否毒化 slab 对象是可以设置的，当 slab 对象被 POISON 之后，那么会有一个问题，就是存放在对象内存区域 object size 里的 freepointer 就被会特殊字节 0x6b 覆盖掉。这种情况下，内核就只能为 freepointer 在额外分配一个 word size 大小的内存空间了。



slab 对象的内存布局信息除了以上内容之外，内核中还使用两个 struct track 结构体来分别存储对象的分配和释放相关信息。

上图展示的就是 slab 对象在内存中的完整布局，其中 object size 为对象真正所需要的内存区域大小，而对象在 slab 中真实的内存占用大小 size 除了 object size 之外，还包括填充的 red zone 区域，以及用于跟踪对象分配和释放信息的 track 结构，另外，如果 slab 设置了 red zone，内核会在对象末尾增加一段 word size 大小的填充 padding 区域。

当 slab 向伙伴系统申请若干内存页之后，内核会按照这个 size 将内存页划分成一个一个的内存块，内存块大小为 size 。其实 slab 的本质就是一个或者多个物理内存页 page，内核会根据上图展示的 slab 对象的内存布局，计算出对象的真实内存占用 size。最后根据这个 size 在 slab 背后依赖的这一个或者多个物理内存页 page 中划分出多个大小相同的内存块出来。

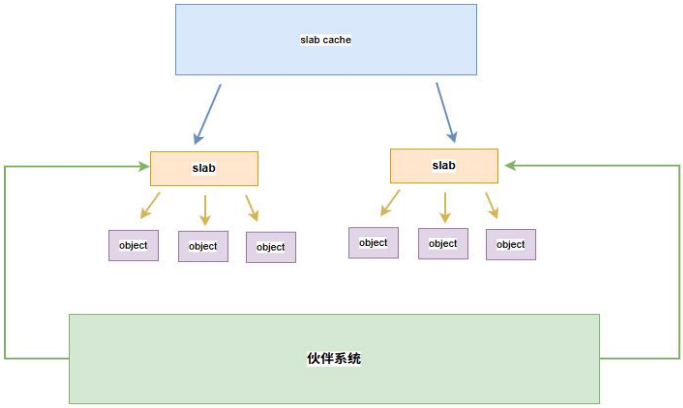
内核中使用结构体 struct slab 来管理 slab 内存池相关的数据。 slab 中直接复用 struct page 的冗余字段实现 struct slab，避免了额外的内存开销。

```
/* Reuses the bits in struct page */
struct slab {
   //复用 struct page 的 flags 字段
    unsigned long __page_flags;
    /*
    指向该 slab 所属的 kmem_cache（slab 缓存描述符），
    关联 slab 到对应的缓存池
    （缓存池定义了对象大小、对齐方式、迁移类型等核心属性），
    是分配 / 释放对象时的核心关联指针。
    */
    struct kmem_cache *slab_cache;
    union {
        struct {
            union {
                // slab 所在的管理链表
                struct list_head slab_list;
#ifdef CONFIG_SLUB_CPU_PARTIAL
                struct {
                   // 用 next 指针在相应管理链表中串联起 slab
                    struct slab *next;
                   // slab 所在管理链表中的包含的 slab 总数
                    int slabs;  /* Nr of slabs left */
                };
#endif
            };
            /* Double-word boundary */
            union {
                struct {
                    // 指向 slab 中第一个空闲对象
                    void *freelist;     /* first free object */
                    union {
```

```
                        unsigned long counters;
                        struct {
                            // slab 中已经分配出去的对像数
                            unsigned inuse:16;
                            unsigned objects:15;// slab 中包含的对象总数


            // 该 slab 是否在对应 slab cache 的本地 CPU 存中
                            unsigned frozen:1;
                        };
                    };
                };
#ifdef system_has_freelist_aba
                freelist_aba_t freelist_counter;
#endif
            };
        };
        struct rcu_head rcu_head;
    };
    unsigned int __unused;// 填充字段：保证内存对齐
    atomic_t __page_refcount;// 复用 struct page 的_refcount 字段
#ifdef CONFIG_MEMCG
    unsigned long memcg_data;  // 内存控制组（memcg）扩展
#endif
};
```

7.5 slab 的总体架构设计

slab 是 slab cache 内存池架构体系中的基本单位，使用 slab 内存池对对象进行分配和释放最终会落在 slab 这个基本单位上。



struct slab 中 list_head 链表节点，用于将 slab 挂入 kmem_cache 的三类核心链表：是 slab 全局管理的核心。

1. full 链表：无空闲对象的 slab。如果一个 slab 中的对象全部分配出去了，slab cache 就会将其视为一个 full slab，表示这个 slab 此刻已经满了，无法在分配对象了。slab cache 就会到伙伴系统中重新申请一个 slab 出来，供后续的内存分配使用。

2. free 链表：全空闲的 slab；当内核将对象释放回其所属的 slab 之后，如果 slab 中的对象全部归位，slab cache 就会将其视为一个 empty slab，表示 slab 此刻变为了一个完全空闲的 slab。如果超过了 slab cache 中规定的 empty slab 的阈值，slab cache 就会将这些空闲的 empty slab 重新释放回伙伴系统中。

3. partial 链表：有部分空闲对象的 slab。如果一个 slab 中的对象部分被分配出去使用，部分却未被分配仍然在 slab 中缓存，那么内核就会将该 slab 视为一个 partial slab。

这三个列表是 slab 全局管理的核心。不同状态的 slab，会在 slab cache 中被不同的链表所管理，同时 slab cache 会控制管理链表中 slab 的个数以及链表中所缓存的空闲对象个数，防止它们无限制的增长。slab cache 中除了需要管理众多的 slab 之外，还包括了很多 slab 的基础信息。比如：

slab 对象内存布局相关的信息。

slab 中的对象需要按照什么方式进行内存对齐，比如，按照 CPU 硬件高速缓存行 cache line (64 字节) 进行对齐，slab 对象是否需要进行毒化 POISON，是否需要在 slab 对象内存周围插入 red zone，是否需要追踪 slab 对象的分配与回收信息，等等。

一个 slab 具体到底需要多少个物理内存页 page，一个 slab 中具体能够容纳多少个 object （内存块）。

7.5.1 slab 的基础信息管理

slab cache 在内核中的数据结构为 struct kmem_cache，里面定了 slab 的基本信息以及 slab 的管理结构全部定义：

```
struct kmem_cache {
#ifndef CONFIG_SLUB_TINY
    struct kmem_cache_cpu __percpu *cpu_slab;
#endif
     // slab cache 的管理标志位，用于设置 slab 的一些特性
     // 比如：slab 中的对象按照什么方式对齐，对象是否需要 POISON  毒化，
    //是否插入 red zone 在对象内存周围，是否追踪对象的分配和释放信息 等等
    slab_flags_t flags;
    //主要控制 NUMA  节点缓存 partial 列表 slab 个数，
    //如果超过该值，那么列表中空闲的 empty slab 就会被释放回伙伴系统中。
    unsigned long min_partial;
    // slab 对象在内存中的真实占用，
    //包括为了内存对齐填充的字节数，red zone 等等
    unsigned int size;
    // slab 中对象的实际大小，不包含填充的字节数
    unsigned int object_size;
    struct reciprocal_value reciprocal_size;
```

```
    // slab 对象池中的对象在没有被分配之前，我们是不关心对象里边存储的内容的。
    //内核巧妙的利用对象占用的内存空间存储下一个空闲对象的地址。
    // offset 表示用于存储下一个空闲对象指针的位置距离对象首地址的偏移
      unsigned int offset;        /* Free pointer offset */
#ifdef CONFIG_SLUB_CPU_PARTIAL
    /* Number of per cpu partial objects to keep around */
    unsigned int cpu_partial;
    /* Number of per cpu partial slabs to keep around */
    unsigned int cpu_partial_slabs;
#endif
      // 表示 cache 中的 slab 大小，
      //包括 slab 所需要申请的页面个数，以及所包含的对象个数
    struct kmem_cache_order_objects oo;

     // 当按照 oo 的尺寸为 slab 申请内存时，如果内存紧张，
      //会采用 min 的尺寸为 slab 申请内存，可以容纳一个对象即可。
    struct kmem_cache_order_objects min;
    // 向伙伴系统申请内存时使用的内存分配标识
    gfp_t allocflags;
    // slab cache 的引用计数，为 0 时就可以销毁并释放内存回伙伴系统重
    int refcount;
   // 池化对象的构造函数，用于创建 slab 对象池中的对象
    void (*ctor)(void *object);

    // 对象的 object_size 按照 word 字长对齐之后的大小
    unsigned int inuse;

     // 对象按照指定的 align 进行对齐
    unsigned int align;
    unsigned int red_left_pad;  /* Left redzone padding size */
     // slab cache 的名称，
    const char *name;       /* Name (only for display!) */
    struct list_head list;      /* List of slab caches */
#ifdef CONFIG_SYSFS
    struct kobject kobj;        /* For sysfs */
#endif
#ifdef CONFIG_SLAB_FREELIST_HARDENED
    unsigned long random;
#endif
#ifdef CONFIG_NUMA
    unsigned int remote_node_defrag_ratio;
#endif
#ifdef CONFIG_SLAB_FREELIST_RANDOM
    unsigned int *random_seq;
```

```
#endif
#ifdef CONFIG_KASAN_GENERIC
    struct kasan_cache kasan_info;
#endif
#ifdef CONFIG_HARDENED_USERCOPY
    unsigned int useroffset;    /* Usercopy region offset */
    unsigned int usersize;      /* Usercopy region size */
#endif
    struct kmem_cache_node *node[MAX_NUMNODES];
};
```

slab_flags_t flags 是 slab cache 的管理标志位，用于设置 slab 的一些特性，比如：当 flags 设置了 **SLAB_HWCACHE_ALIGN** 时，表示 slab 中的对象需要按照 CPU 硬件高速缓存行 cache line (64 字节) 进行对齐。

当 flags 设置了 **SLAB_POISON** 时，表示需要在 slab 对象内存中填充特殊字节 0x6b 和 0xa5，表示对象的特定状态。

当 flags 设置了 **SLAB_RED_ZONE** 时，表示需要在 slab 对象内存周围插入 red zone，防止内存的读写越界。

当 flags 设置了 **SLAB_CACHE_DMA** 或者 **SLAB_CACHE_DMA32** 时，表示指定 slab 中的内存来自于哪个内存区域，DMA or DMA32 区域 ？如果没有特殊指定，slab 中的内存一般来自于 NORMAL 直接映射区域。

当 flags 设置了 **SLAB_STORE_USER** 时，表示需要追踪对象的分配和释放相关信息，这样会在 slab 对象内存区域中额外增加两个 sizeof(struct track) 大小的区域出来，用于存储 slab 对象的分配和释放信息。

相关 slab cache 的标志位 flag，定义在内核文件 include\linux\slab.h 中：

```
/* DEBUG: Perform (expensive) checks on alloc/free */
#define SLAB_CONSISTENCY_CHECKS ((slab_flags_t __force)0x00000100U)
/* DEBUG: Red zone objs in a cache */
#define SLAB_RED_ZONE       ((slab_flags_t __force)0x00000400U)
/* DEBUG: Poison objects */
#define SLAB_POISON     ((slab_flags_t __force)0x00000800U)
/* Indicate a kmalloc slab */
#define SLAB_KMALLOC        ((slab_flags_t __force)0x00001000U)
/* Align objs on cache lines */
#define SLAB_HWCACHE_ALIGN  ((slab_flags_t __force)0x00002000U)
/* Use GFP_DMA memory */
#define SLAB_CACHE_DMA      ((slab_flags_t __force)0x00004000U)
/* Use GFP_DMA32 memory */
#define SLAB_CACHE_DMA32    ((slab_flags_t __force)0x00008000U)
/* DEBUG: Store the last owner for bug hunting */
#define SLAB_STORE_USER     ((slab_flags_t __force)0x00010000U)
```

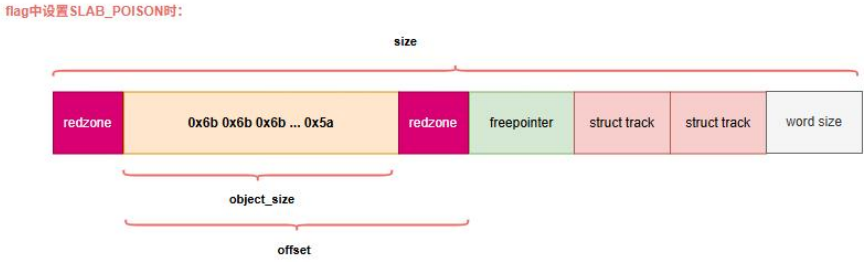
size 字段表示 slab 对象在内存中的真实占用大小，该大小包括对象所占内存中各种填充的内存区域大小。

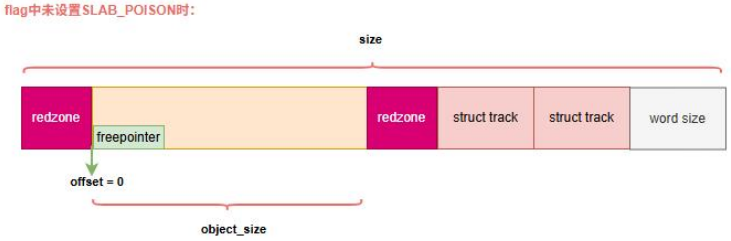
unsigned int object_size 表示单纯的存储 slab 对象所需要的实际内存大小。

当 kmem_cache 结构中的 flags 设置了 SLAB_POISON 标志位之后，slab 中的对象会

POISON 毒化，被特殊字节 0x6b 和 0xa5 所填充，这样一来就会覆盖原有的 freepointer，在这种情况下，内核就需要把 freepointer 存储在对象所在内存区域的外面。所以内核就需要用一个字段来标识 freepointer 的位置，struct kmem_cache 结构中的 **unsigned int offset** 字段就表示对象的 freepointer 指针距离对象的起始内存地址的偏移。

上述介个变量的关系，如下如图所示。





struct kmem_cache_order_objects oo 字段就是保存了一个 slab 中到底包含多少个内存页，以及这些内存页中到底能容纳多少个内存块（object）。struct kmem_cache_order_objects 结构体其实就是一个无符号的整形字段，它的高 16 位用来存储 slab 所需的物理内存页个数，低 16 位用来存储 slab 所能容纳的对象总数。

```
struct kmem_cache_order_objects {
    unsigned int x;
};
```

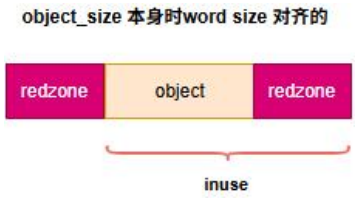
struct kmem_cache_order_objects max 字段表示 oo 的最大值，内核在初始化 slab 的时候，会将 max 的值设置为 oo。

struct kmem_cache_order_objects min 字段表示 slab 中至少需要容纳的对象个数以及容纳最少的对象所需要的内存页个数。内核在初始化 slab 的时候会 将 min 的值设置为至少需要容纳一个对象。

内核在创建 slab 的时候，最开始会按照 oo 指定的尺寸来向伙伴系统申请内存页，如果内存紧张，申请内存失败。那么内核会降级采用 min 的尺寸再次向伙伴系统申请内存。也就是说 slab 中至少会包含一个对象。

gfp_t allocflags 是内核在向伙伴系统为 slab 申请内存页的时候，所用到的内存分配标志位。

unsigned int inuse 表示对象的 object size 按照 word size 对齐之后的大小，如果设置了 SLAB_RED_ZONE，inuse 也会包括对象右侧 red zone 区域的大小。

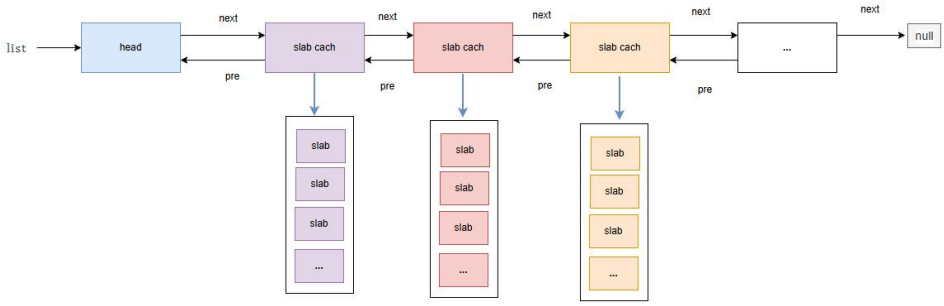


unsigned int align 在创建 slab cache 的时候，我们可以向内核指定 slab 中的对象按照 align 的值进行对齐，内核会综合 word size , cache line ，align 计算出一个合理的对齐尺寸。

const char *name 表示该 slab cache 的名称。

内核会将系统中的所有 slab cache 用一个双向链表统一串联起来。链表的头结点指针保存在 struct kmem_cache 结构的 list 中。

```
struct kmem_cache {
    ......
    // 用于组织串联系统中所有类型的 slab cache
    struct list_head list;
    ......
}
```



7.5.2 slab 的组织架构

slab cache 其实就是内核中的一个对象池。内核对 slab cache 的设计充分考虑了多进程并发访问 slab cache 所带来的同步性能开销，内核在 slab cache 的设计中为每个 cpu 引入了 struct kmem_cache_cpu 结构的 percpu 变量，作为 slab cache 在每个 cpu 中的本地缓存。

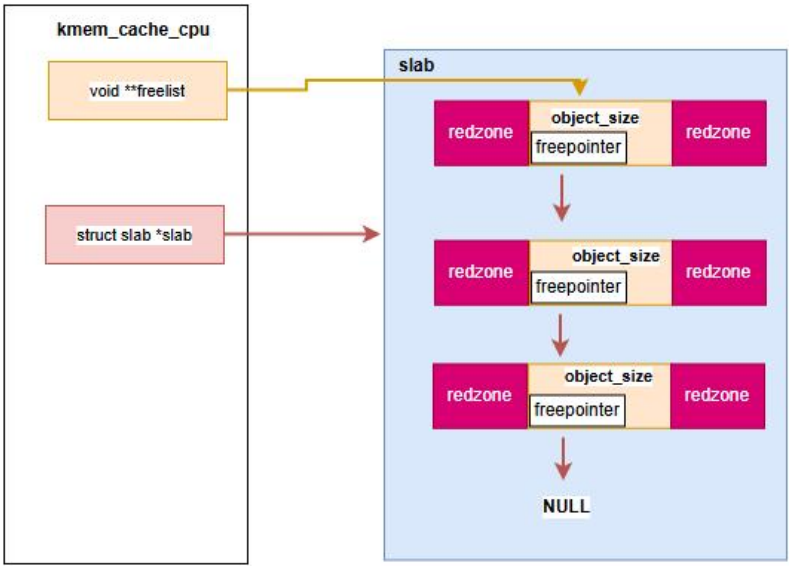
```
struct kmem_cache {

    // 每个 cpu 拥有一个本地缓存，用于无锁化快速分配释放对象
    struct kmem_cache_cpu __percpu *cpu_slab;
    ......

}
```

当进程需要向 slab cache 申请对应的内存块（object）时，首先会直接来到 kmem_cache_cpu 中查看 cpu 本地缓存的 slab，如果本地缓存的 slab 中有空闲对象，那么就直接返回了，整个过程完全没有加锁。而且访问路径特别短，防止了对 CPU 硬件高速缓存 L1Cache 中的 Instruction Cache（指令高速缓存）污染。

cpu 本地缓存 kmem_cache_cpu 结构的具体内容：

```
struct kmem_cache_cpu {
    union {
        struct {
            // 指向被 CPU 本地缓存的 slab 中第一个空闲的对象
            void **freelist;
            // 保证进程在 slab cache 中获取到的
            //cpu 本地缓存 kmem_cache_cpu 与当前执行进程的 cpu 是一致的。
            unsigned long tid;
        };
        freelist_aba_t freelist_tid;
    };
    //当前 CPU 绑定的 slab（处于 frozen=1 状态），
    //CPU 优先从该 slab 提取对象补充本地 freelist。
    struct slab *slab;  /* The slab from which we are allocating */
#ifdef CONFIG_SLUB_CPU_PARTIAL
//CPU 私有 partial 链表，存储“已冻结但未完全分配”的 slab，优先级高于全局
//partial 链表，进一步提升本地分配效率。
    struct slab *partial;   /* Partially allocated frozen slabs */
#endif
    local_lock_t lock;  /* Protects the fields above */
#ifdef CONFIG_SLUB_STATS
    // 记录 slab 分配对象的一些状态信息
    unsigned int stat[NR_SLUB_STAT_ITEMS];
#endif
};
```

freelist 指针指向的是该 slab 中第一个空闲的对象。为了充分利用 slab 对象所占用的内存，内核会在对象占用内存区域内开辟一块区域来存放 freepointer 指针，而 freepointer 可以用来指向下一个空闲对象。这样一来，通过这里的 freelist 和 freepointer 就将 slab 中所有的空闲对象串联了起来。



kmem_cache_cpu 结构中的 tid 是内核为 slab cache 的 cpu 本地缓存结构设置的一个全局唯一的 transaction id ，这个 tid 在 slab cache 分配内存块的时候主要有两个作用：

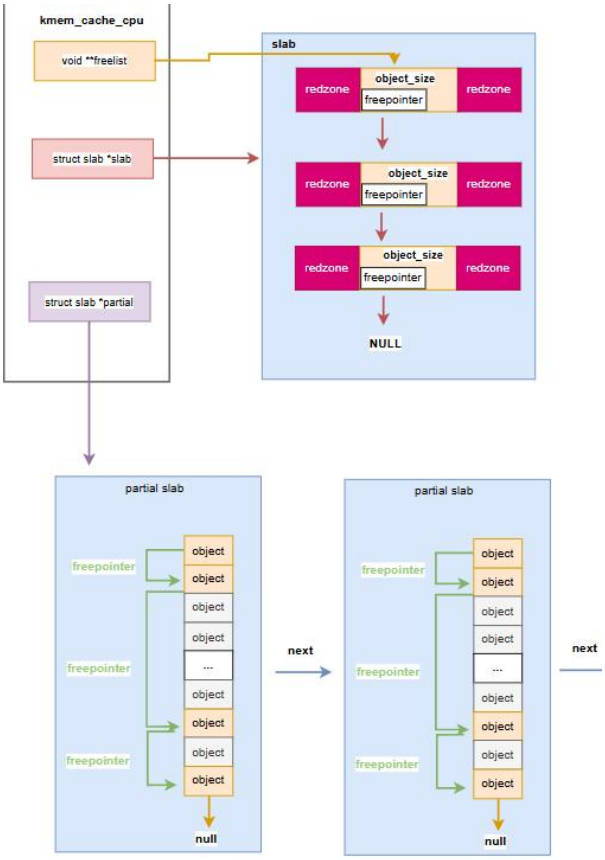
1. 内核会将 slab cache 每一次分配内存块或者释放内存块的过程视为一个事务，所以在每次向 slab cache 申请内存块或者将内存块释放回 slab cache 之后，内核都会改变这里的 tid。

2. tid 也可以简单看做是 cpu 的一个编号，每个 cpu 的 tid 都不相同，可以用来标识区分不同 cpu 的本地缓存 kmem_cache_cpu 结构。

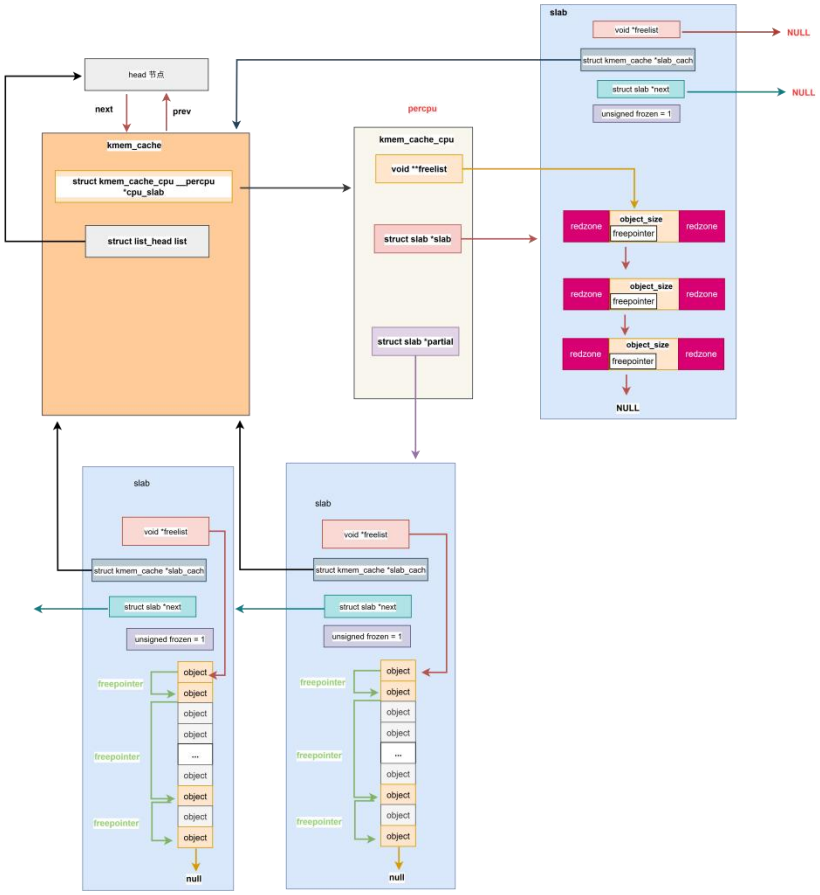
其中， tid 的第二个作用是最主要的，因为进程可能在执行的过程中被更高优先级的进程抢占 cpu （开启 CONFIG_PREEMPT 允许内核抢占）或者被中断，随后进程可能会被内核重新调度到其他 cpu 上执行，这样一来，进程在被抢占之前获取到的 kmem_cache_cpu 就与当前执行进程 cpu 的 kmem_cache_cpu 不一致了。

如果开启了 CONFIG_SLUB_CPU_PARTIAL 配置项，那么在 slab cache 的 cpu 本地缓存 kmem_cache_cpu 结构中就会多出一个 partial 列表，partial 列表中存放的都是 partial slub，相当于是 cpu 缓存的备用选择。

当 kmem_cache_cpu->slab（被本地 cpu 所缓存的 slab）中的对象已经全部分配出去之后，内核会到 partial 列表中查找一个 partial slab 出来，并从这个 partial slab 中分配一个对象出来，最后将 kmem_cache_cpu->slab 指向这个 partial slab，作为新的 cpu 本地缓存 slab。这样一来，下次分配对象的时候，就可以直接从 cpu 本地缓存中获取了。



内核中使用数据结构 struct slab 描述单个 slab；使用数据结构 struct kmem_cache 描述 slab cache ；使用数据结构 struct kmem_cache_cpu 描述 slab cache 的本地 cpu 缓。将这种三种数据结构结合起来，就可以得到 slab cache 大致的架构图：



但这还不是 slab cache 的最终架构。 如果 slab cache 结构比作大型超市，那么一个 kmem_cache 结构实体就好比一个具体的超市，slab cache 的本地 cpu 缓存结构 kmem_cache_cpu 就好比超市的营业厅，slab 就好比一排一排的货架， slab 中划分出来的一个一个的内存块就好比货架上的商品。

slab cache 结构中的内存块，实际来自于 NUMA 节点中。每个 cpu 都可以任意访问所有 NUMA 节点中的内存，只是访问速度上有差异， cpu 在访问本地 NUMA 节点的速度是最快的，当本地 NUMA 节点中的内存不足时，cpu 会跨节点访问其他 NUMA 节点。

通过 kmem_cache_cpu 中的 slab 字段，与具体的 NUMA 节点建立联系。从 slab cache 获取内存块时，首先会从本地 cpu 缓存 kmem_cache_cpu 中分配，此时 slab 指向本地 CPU 所属的 NUMA 节点中的物理页，若本地 cpu 缓存 kmem_cache_cpu 中没有足够的内存块可供分配时，内核就会从 struct kmem_cache_node *node[MAX_NUMNODES]数组中的某一个 numa 节点中拿出 slab 填充到 kmem_cache_cpu 中，此时 slab 指这个节点中的内存页。

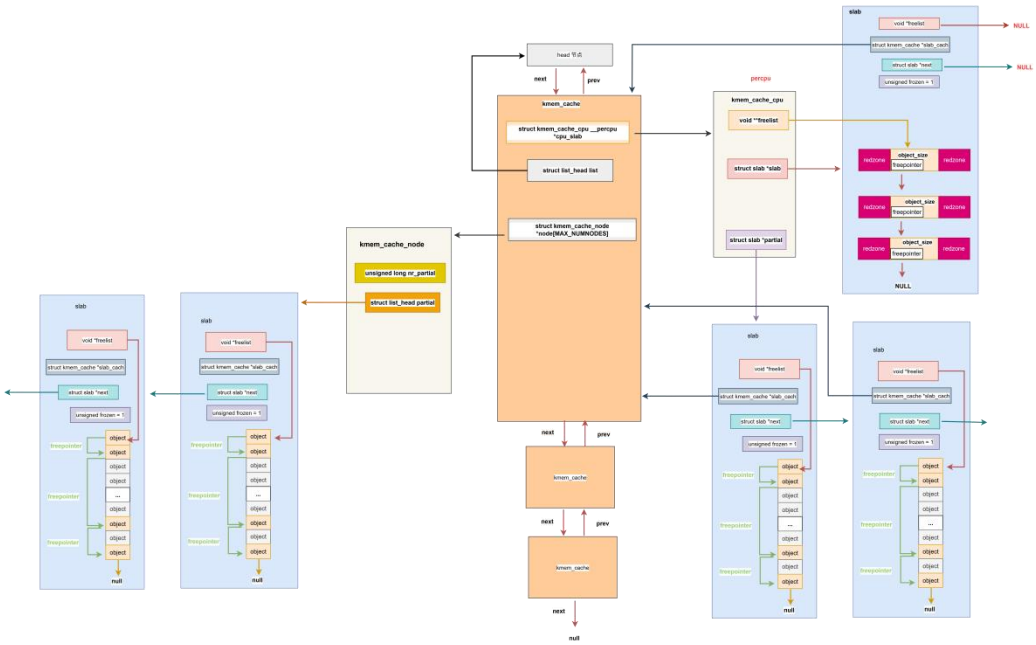
```
struct kmem_cache_cpu {
    ......
    struct slab *slab;
    ....

}
```

slab cache 中的 struct kmem_cache_node 结构如下：

```
struct kmem_cache {
      ......
      //slab cache 中 numa node 中的缓存，每个 node 一个
    struct kmem_cache_node *node[MAX_NUMNODES];
    ......
}
```

```
struct kmem_cache_node {
    spinlock_t list_lock;
    // 该 node 节点中缓存的 slab 个数
    unsigned long nr_partial;
    // 该链表用于组织串联 node 节点中缓存的 slabs
    // partial 链表中缓存的 slab 为部分空闲的（slab 中的对象部分被分配出去）
    struct list_head partial;
    // slab 的个数
    atomic_long_t nr_slabs;
     // 该 node 节点中缓存的所有 slab 中包含的对象总和
    atomic_long_t total_objects;
    // full 链表中包含的 slab 全部是已经被分配完毕的 full slab
    struct list_head full;
    ......
};
```

补充了 struct kmem_cache_node 结构之后，lab cache 的架构全貌，如下图所示：



其中，slab cache 本地 cpu 缓存 kmem_cache_cpu 中的 partial 列表以

及 NUMA 节点缓存 kmem_cache_node 结构中的 partial 列表并不是无限制增长的，它们的容量收到下面两个参数的限制：

```
struct kmem_cache {
    ......
    // slab cache 在 numa node 中缓存的 slab 个数上限，
    //slab 个数超过该值，空闲的 empty slab 则会被回收至伙伴系统
    unsigned long min_partial;
    ......
#ifdef CONFIG_SLUB_CPU_PARTIAL
     // 限定 slab cache 在每个 cpu 本地缓存 partial 链表中
      //所有 slab 中空闲对象的总数
    unsigned int cpu_partial;
    // 限定 slab cache 在每个 cpu 本地缓存 partial 链表中
    //所有 slab 的总数
    unsigned int cpu_partial_slabs;

#endif


}
```

min_partial 主要控制 NUMA 节点缓存的 slab 个数上线，如果超过该值，那么列表中空闲的 empty slab 就会被释放回伙伴系统中。

cpu_partial 主要控制 slab cache 本地 cpu 缓存 kmem_cache_cpu 结构 partial 链表中缓存的空闲对象总数，如果超过该值，那么 kmem_cache_cpu->partial 列表中缓存的 slab 将会被全部转移至 kmem_cache_node->partial 列表中。

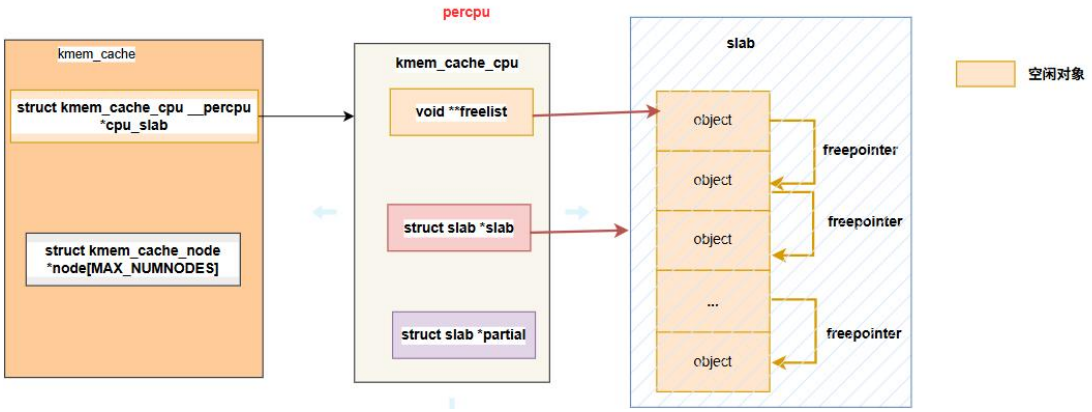
cpu_partial_slabs 限定 slab cache 在每个 cpu 本地缓存 partial 链表中所有 slab 的总数

7.6 slab 内存分配原理

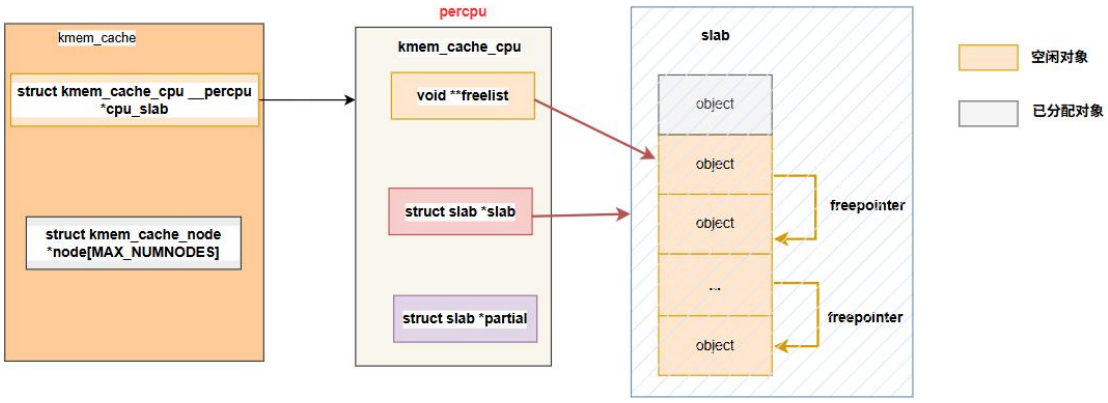
同伙伴系统的内存分配原理一样，slab cache 在分配内存块的时候同样也分为快速路径 fastpath 和慢速路径 slowpath，但 slab cache 的组织架构比较复杂，所以在分配内存块的时候又会分为很多场景。

7.6.1 从本地 cpu 缓存中直接分配

slab cache 的本地 cpu 缓存中的有很多的空闲对象时，当内核向该 slab cache 申请对象的时候，首先会进入快速分配路径 fastpath，通过 kmem_cache_cpu->freelist 直接查看本地 cpu 缓存 kmem_cache_cpu->slab 中是否有空闲对象可供分配。

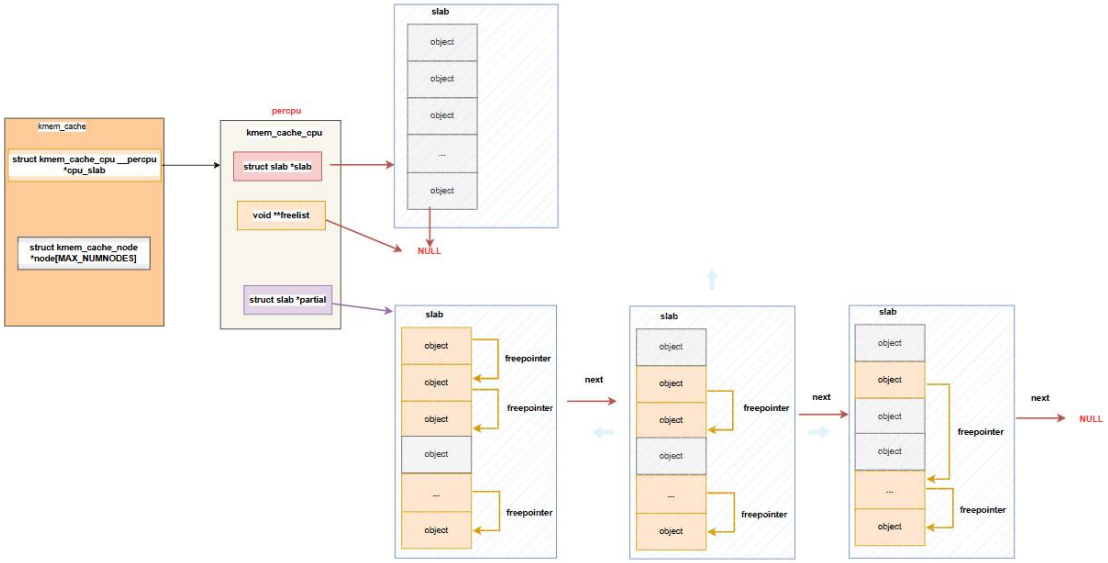


如果有，则将 kmem_cache_cpu->freelist 指向的第一个空闲对象拿出来分配，随后调整 kmem_cache_cpu->freelist 指向下一个空闲对象。

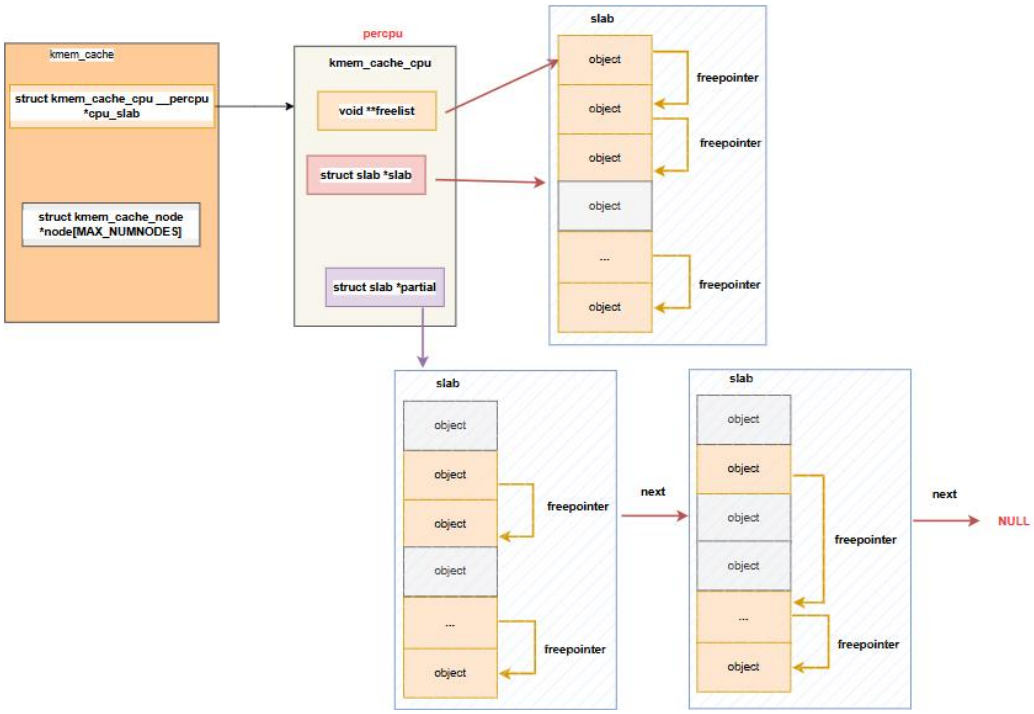


7.6.3 从本地 cpu 缓存 partial 列表中分配

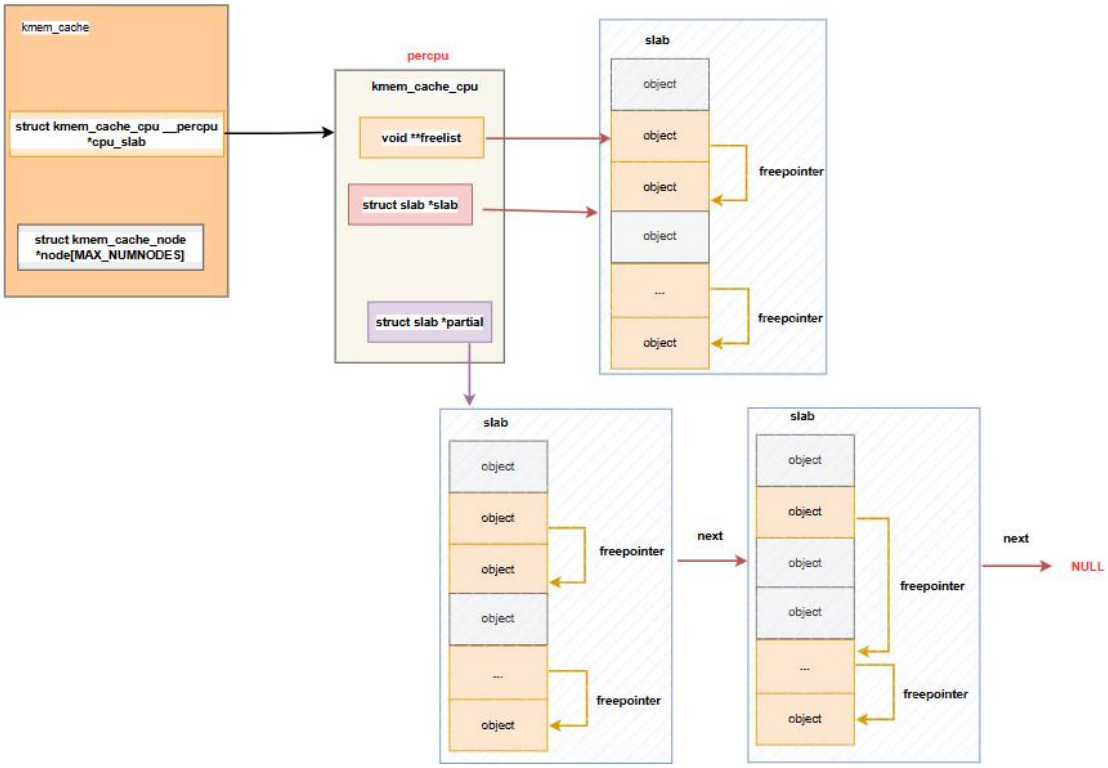
当 slab cache 本地 cpu 缓存的 slab (kmem_cache_cpu->slab) 中没有任何空闲的对象时（全部被分配出去了），那么 slab cache 的内存分配就会进入慢速路径 slowpath。内核会到本地 cpu 缓存的 partial 列表中去查看是否有一个 slab 可以分配对象。这里内核会从 partial 列表中的头结点开始遍历直到找到一个可以满足分配的 slab 出来。



随后内核会将该 slab 从 partial 列表中摘下，直接提升为新的本地 cpu 缓存。

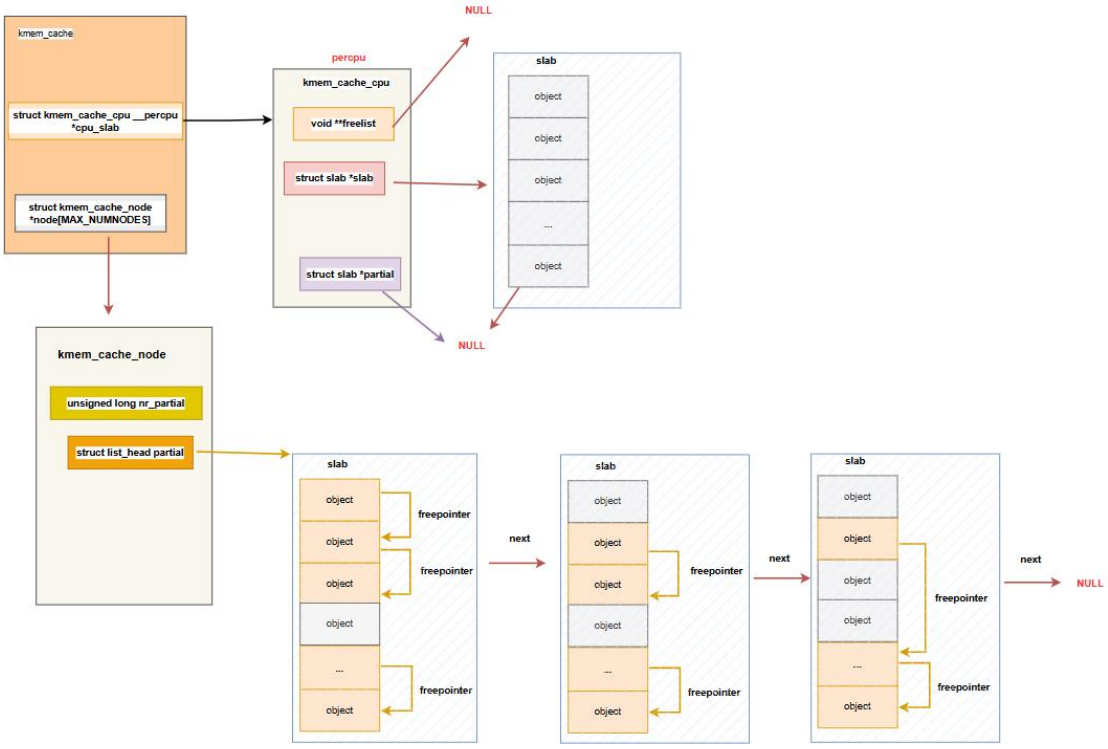


这样一来 slab cache 的本地 cpu 缓存就被更新了，内核通过 kmem_cache_cpu->freelist 指针将缓存 slab 中的第一个空闲对象分配出去，随后更新 kmem_cache_cpu->freelist 指向 slab 中的下一个空闲对象。

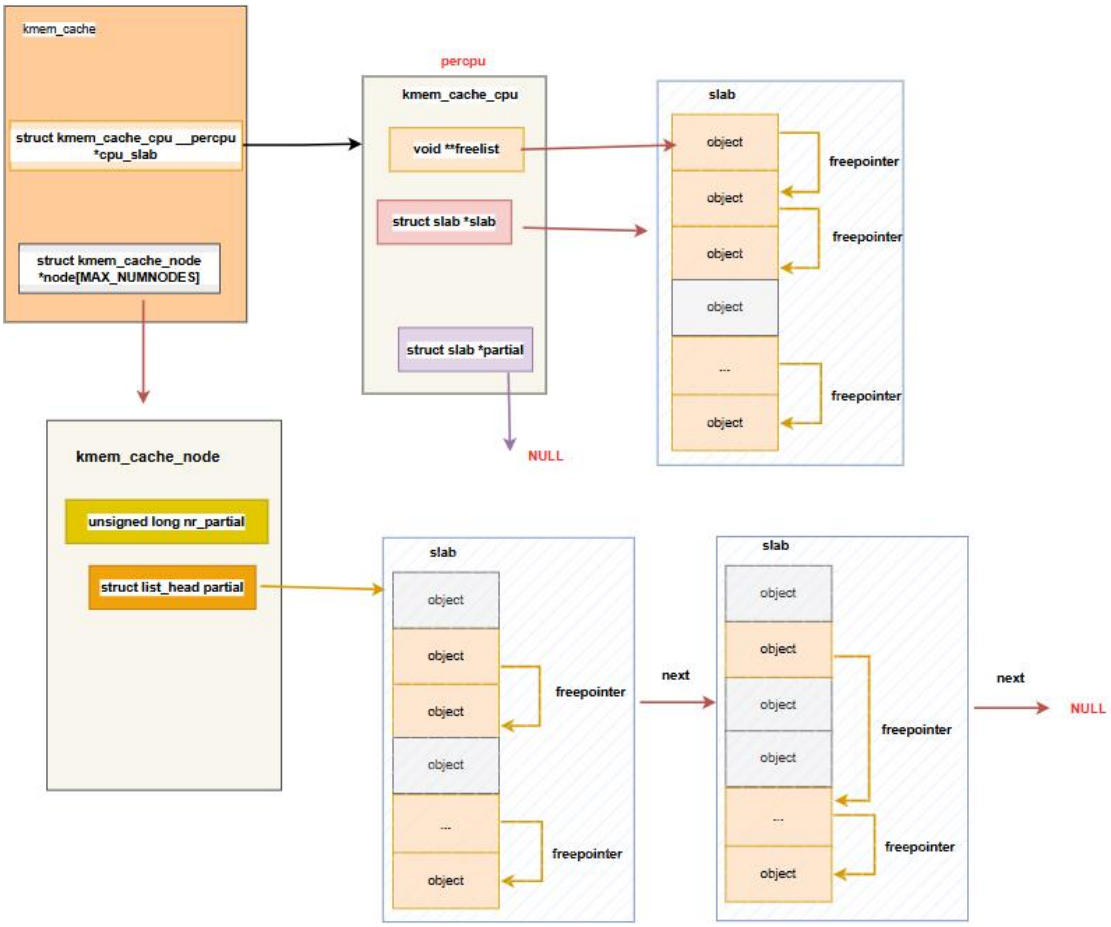


7.6.4 从 NUMA 节点缓存中分配

随着时间的推移， slab cache 本地 cpu 缓存的 slab 中的对象被一个一个的分配出去，变成了一个 full slab，于此同时本地 cpu 缓存 partial 链表中的 slab 也被全部摘除完毕，此时是一个空的链表。此时 slab cache 就该从 kmem_cache_node 结构中的 partial 链表中的 slab 分配对象。

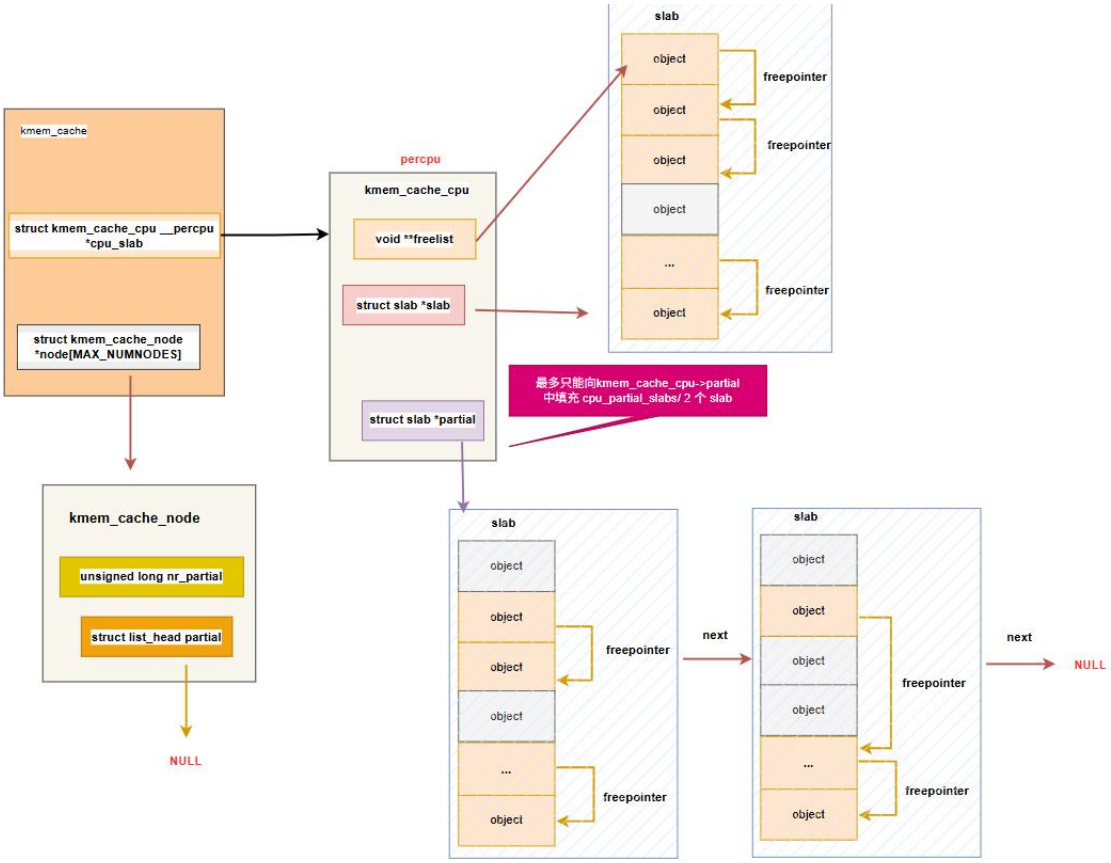


内核会从 kmem_cache_node->partial 链表的头结点开始遍历，将遍历到的第一个 slab 从链表中摘下，直接提升为新的本地 cpu 缓存 kmem_cache_cpu->slab，kmem_cache_cpu->freelist 指针重新指向该 slab 中第一个空闲独享。

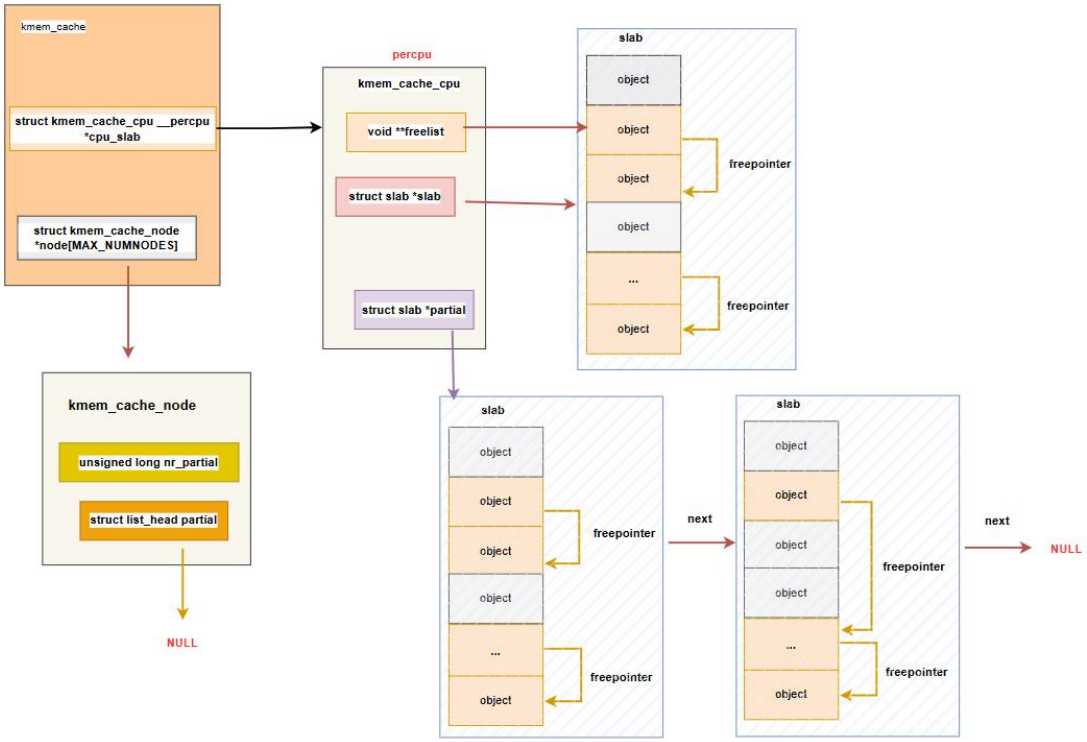


随后内核会接着遍历 kmem_cache_node->partial 链表，将链表中的 slab 挨个摘下填充到本地 cpu 缓存 partial 链表中。最多只能填充 cpu_partial_slabs/ 2 个 slab。

```
struct kmem_cache {
    ......
    // 限定 slab cache 在每个 cpu 本地缓存 partial 链表中
    //所有 slab 的总数
    unsigned int cpu_partial_slabs;

    ......

}
```

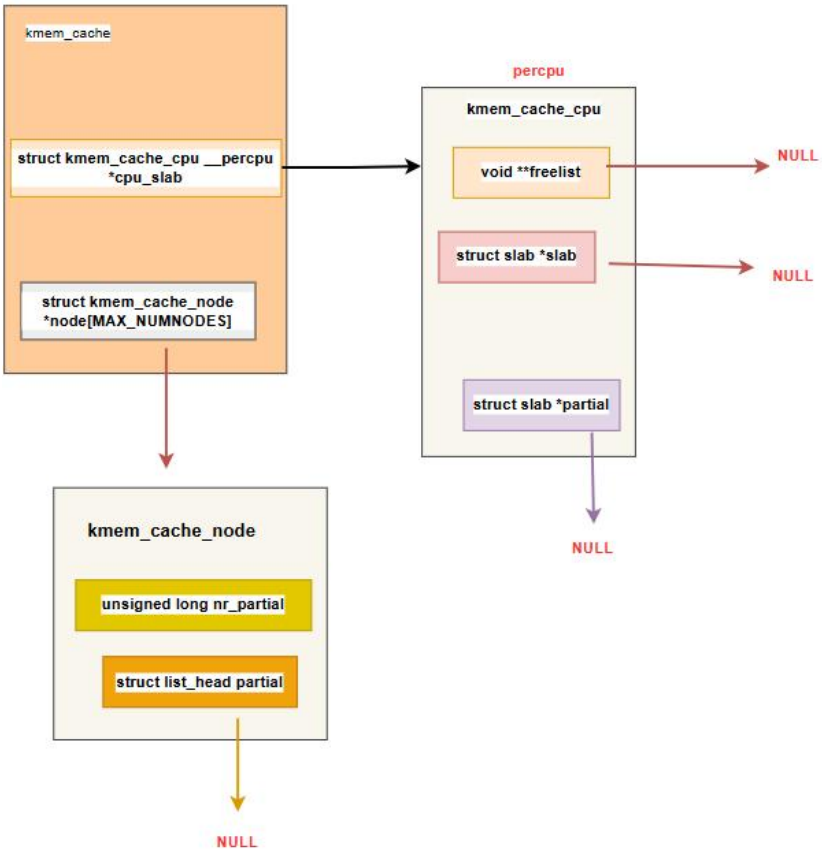


slab cache 就从仓库 kmem_cache_node->partial 链表中重新填充了本地 cpu 缓存 kmem_cache_cpu->page 以及 kmme_cache_cpu->partial 链表。随后内核直接从本地 cpu 缓存中，通过 kmem_cache_cpu->freelist 指针将缓存 slab 中的第一个空闲对象分配出去，随后更新 kmem_cache_cpu->freelist 指向 slab 中的下一个空闲对象。



7.6.5 从伙伴系统中重新申请 slab

当 slab cache 的本地 cpu 缓存 kmem_cache_cpu->page 是空的，kmem_cache_cpu->partial 链表中也是空，NUMA 节点缓存 kmem_cache_node->partial 链表中也是空的时候，比如，slab cache 在刚刚被创建出来时。

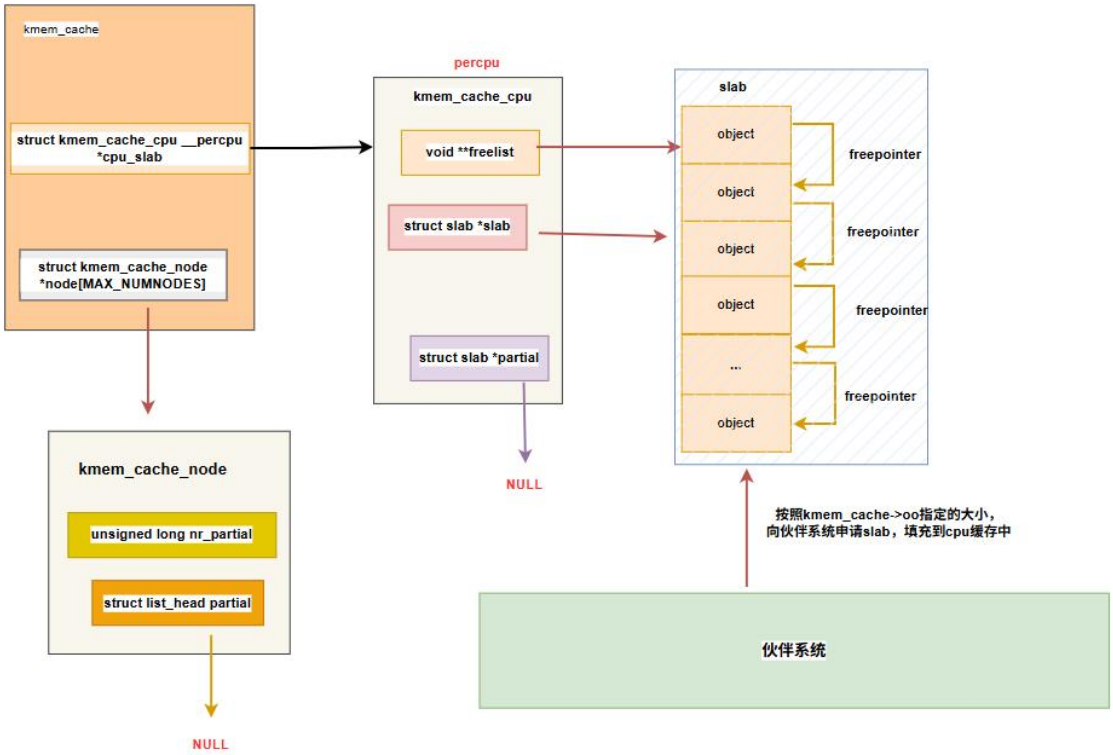


这时，内核就需要到伙伴系统中重新申请一个 slab 出来，具体向伙伴系统申请多少内存页是由 struct kmem_cache 结构中的 oo 来决定的，它的高 16 位表示一个 slab 所需要的内存页个数，低 16 位表示 slab 中所包含的对象总数。

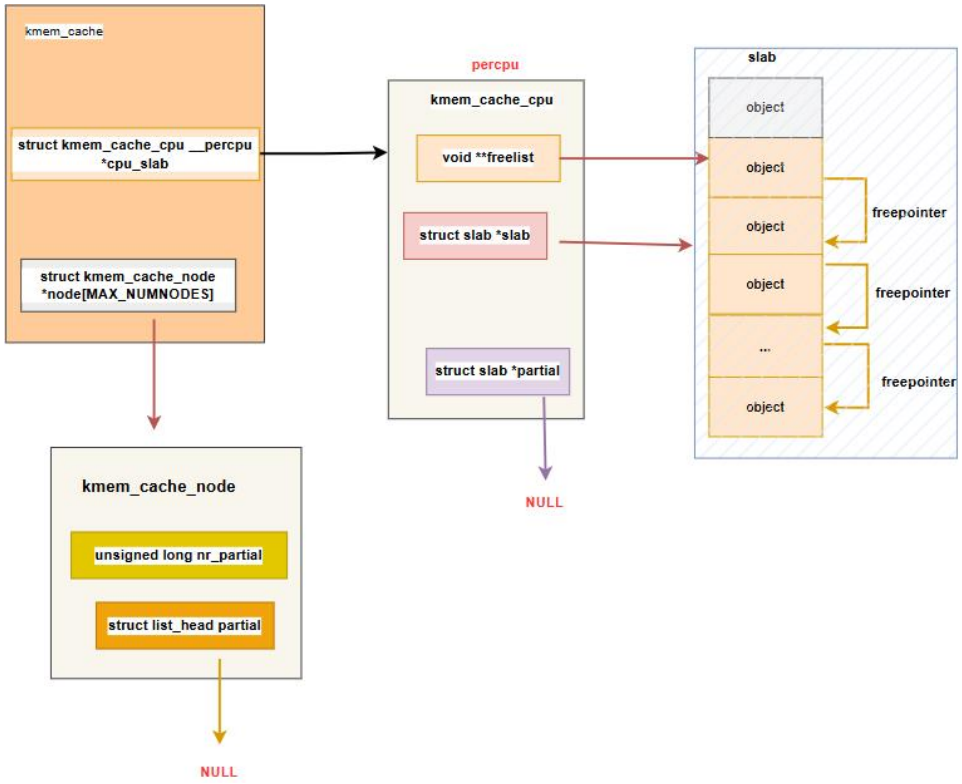
```
struct kmem_cache {

    ......
    // 表示 cache 中的 slab 大小，包括 slab 所申请的页面个数，
    //以及所包含的对象个数其中低 16 位表示一个 slab 中所包含的对象总数，
    //高 16 位表示一个 slab 所占有的内存页个数。
    struct kmem_cache_order_objects oo;
    // 当按照 oo 的尺寸为 slab 申请内存时，如果内存紧张，
    //会采用 min 的尺寸为 slab 申请内存，可以容纳一个对象即可。
    struct kmem_cache_order_objects min;
```

```
    ......

}
```

当系统中空闲内存不足时，无法获得 oo 指定的内存页个数，那么内核会降级采用 min 指定的内存页个数，重新到伙伴系统中去申请。当内核从伙伴系统中申请出指定的内存页个数之后，初始化 slab，最后将初始化好的 slab 直接提升为本地 cpu 缓存 kmem_cache_cpu->slab 。



slab cache 的本地 cpu 缓存被重新填充后，内核直接从本地 cpu 缓存中，通过 kmem_cache_cpu->freelist 指针将缓存 slab 中的第一个空闲对象分配出去，随后更新 kmem_cache_cpu->freelist 指向 slab 中的下一个空闲对象。

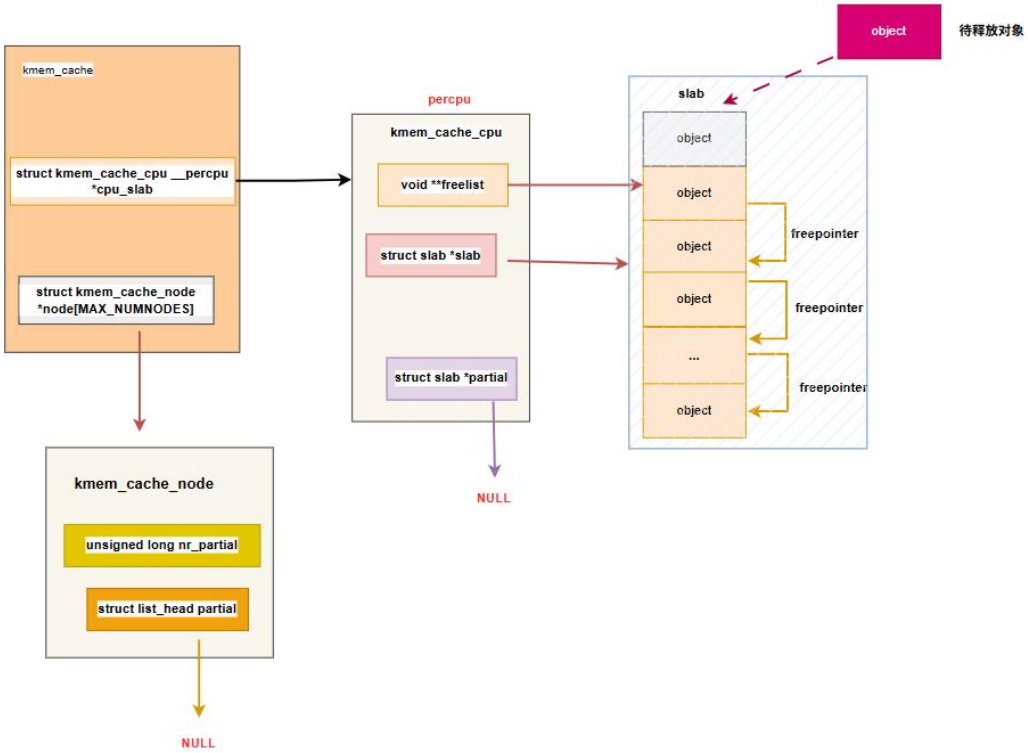


7.7 slab 内存释放原理

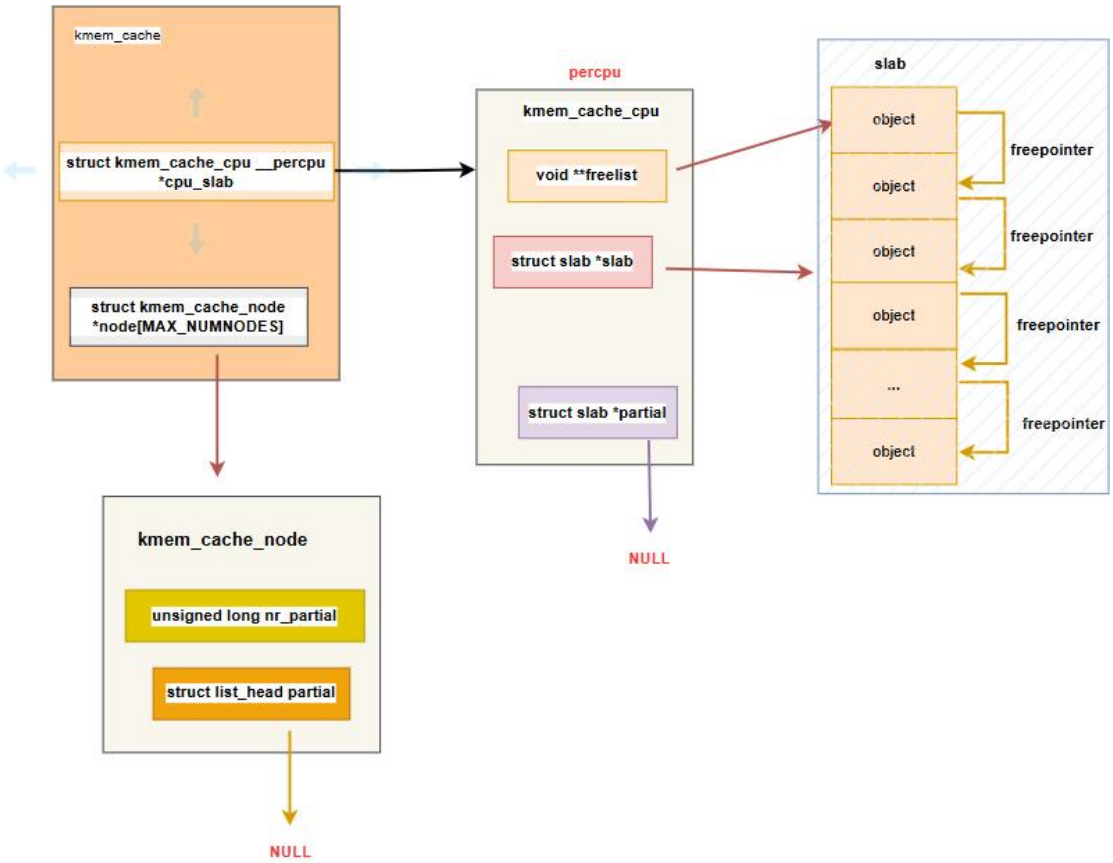
slab cache 的内存释放正好和内存分配的过程相反，但内存释放的过程会比内存分配的过程复杂一些，内存释放同样也包含快速路径 fastpath 和慢速路径 slowpath，也会分为很多场景。

7.7.1 释放对象所属 slab 在 cpu 本地缓存中

如果将要释放回 slab cache 的对象所在的 slab 刚好是本地 cpu 缓存中缓存的 slab，那么内核直接会把对象释放回缓存的 slab 中，这个就是 slab cache 的快速内存释放路径 fastpath。



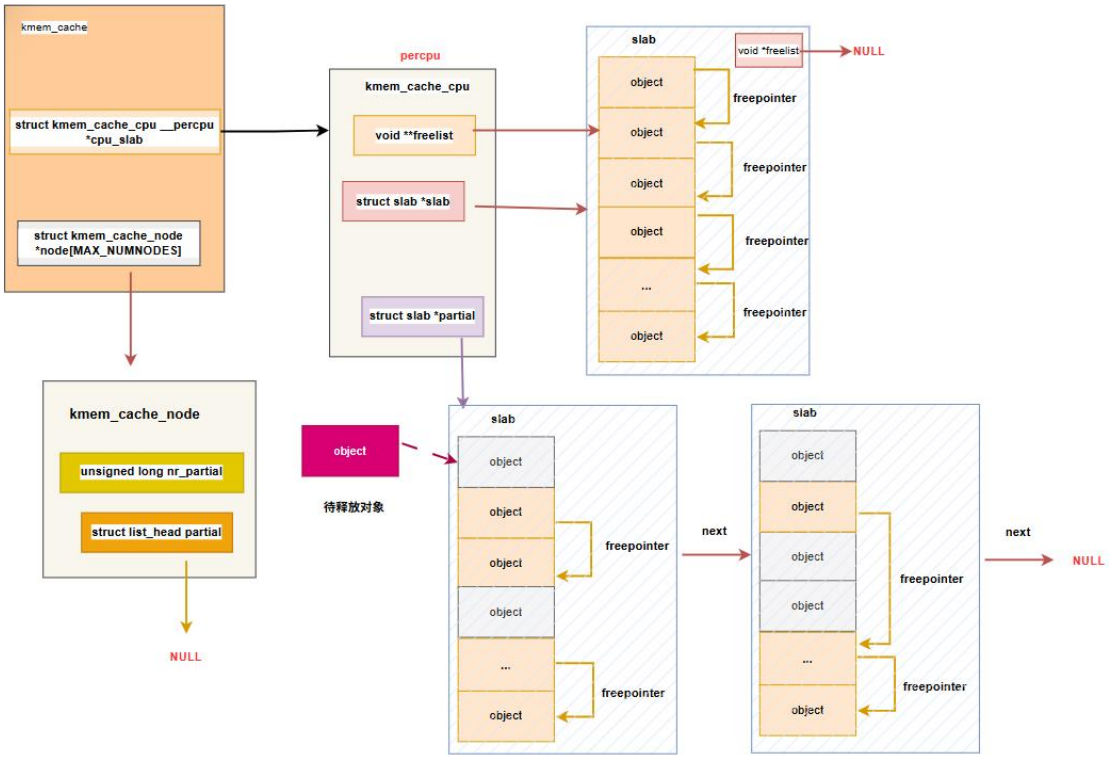
随后修正 kmem_cache_cpu->freelist 指针使其指向刚刚被释放的对象，释放对象的 freepointer 指针指向原来 kmem_cache_cpu->freelist 指向的对象。



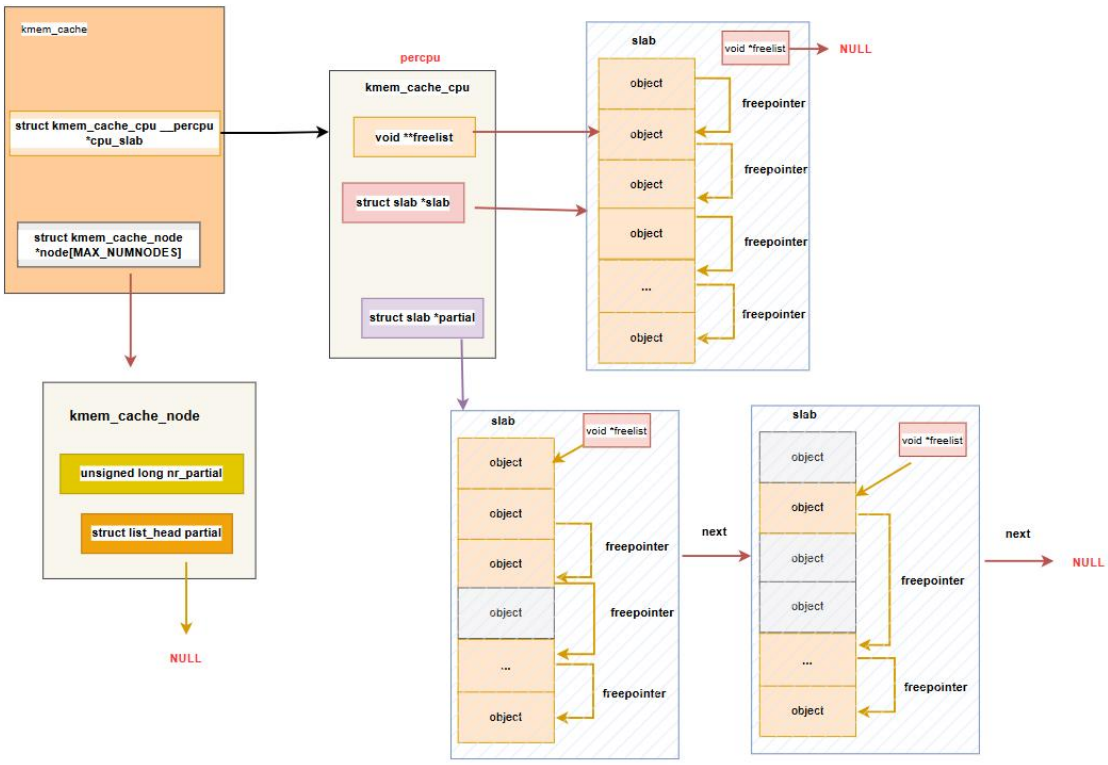
7.7.2 释放对象所属 slab 在 cpu 本地缓存 partial 列表中

当释放的对象所属的 slab 在 cpu 本地缓存 kmem_cache_cpu->partial 链表中时，内核也是直接将对象释放回 slab 中，然后修改 slab （struct page）中的 freelist 指针指向刚刚被释放的对象。释放对象的 freepointer 指向其下一个空闲对象。

释放前：



释放后：



7.7.3 释放对象所属 slab 从 full slab 变为了 partial slab

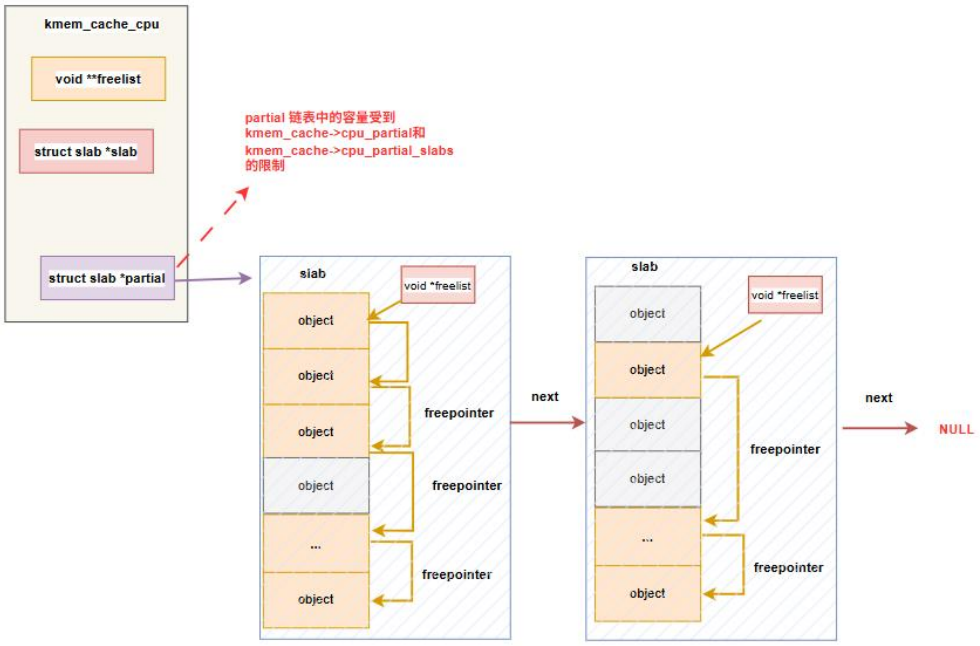
当释放对象所在的 slab 原来是一个 full slab，由于对象的释放刚好变成了一个 partial slab，并且该 slab 原来并不在 slab cache 的本地 cpu 缓存中。这种情况下，当对象释放回 slab 之后，内核为了利用局部性的优势需要把该 slab 在插入到 slab cache 的本地 cpu 缓存 kmem_cache_cpu->partial 链表中。

因为 slab 之前之所以是一个 full slab，恰恰证明了该 slab 是一个非常活跃的 slab，常常供不应求导致变成了一个 full slab，当对象释放之后，刚好变成 partial slab，这时需要将这个被频繁访问的 slab 放入 cpu 缓存中，加快下次分配对象的速度。

但是 slab cache 的本地 cpu 缓存 kmem_cache_cpu->partial 链表中的容量不可能是无限制增长的，它受到 kmem_cache 结构中 cpu_partial 属性的限制：

```
struct kmem_cache {
    ......

    // 限定 slab cache 在每个 cpu 本地缓存 partial 链表中
    //所有 slab 的总数
    unsigned int cpu_partial_slabs;

    // 限定 slab cache 在每个 cpu 本地缓存 partial 链表中
    //所有 slab 中空闲对象的总数
     unsigned int cpu_partial;

    ......
```

```
}
```



当每次向 kmem_cache_cpu->partial 链表中填充 slab 的时候，内核都需要首先检查当前 kmem_cache_cpu->partial 链表中所有 slabs 所包含的空闲对象总数是否超过了 cpu_partial 的限制。如果没有超过限制，则将 slab 插入到 kmem_cache_cpu->partial 链表的头部，如果超过了限制，则需要首先将当前 kmem_cache_cpu->partial 链表中的所有 slab 转移至对应的 NUMA 节点缓存 kmem_cache_node->partial 链表的尾部，然后才能将释放对象所在的 slab 插入到 kmem_cache_cpu->partial 链表中。

把 kmem_cache_cpu->partial 链表中的 slab 一次性全部移动到 kmem_cache_node->partial 链表中的原因？

在内存频繁释放的场景下，kmem_cache_cpu->partial 链表太满了，超过了 cpu_partial 的限制。而内存分配的请求又不是很多，kmem_cache_cpu 中缓存的 slab 并不会频繁的消耗。这样一来，就需要将链表中的所有 slab 一次性转移到 NUMA 节点缓存 partial 链表中备用。否则的话，就得频繁的转移 slab，这样性能消耗更大。

7.7.4 释放对象所属 slab 从 full slab 变为了 partial slab

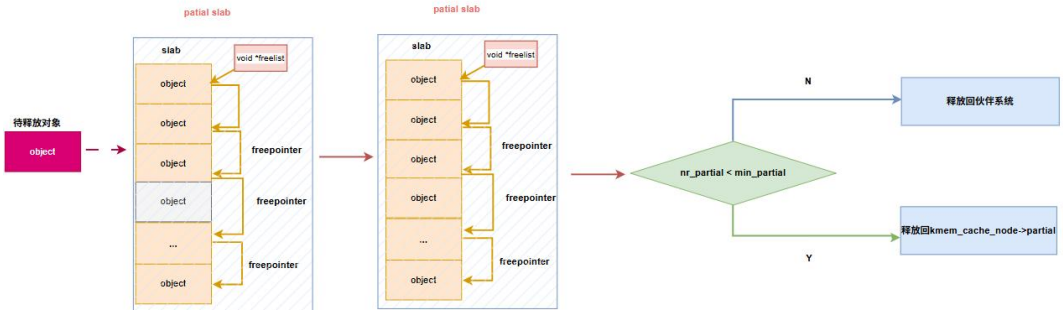
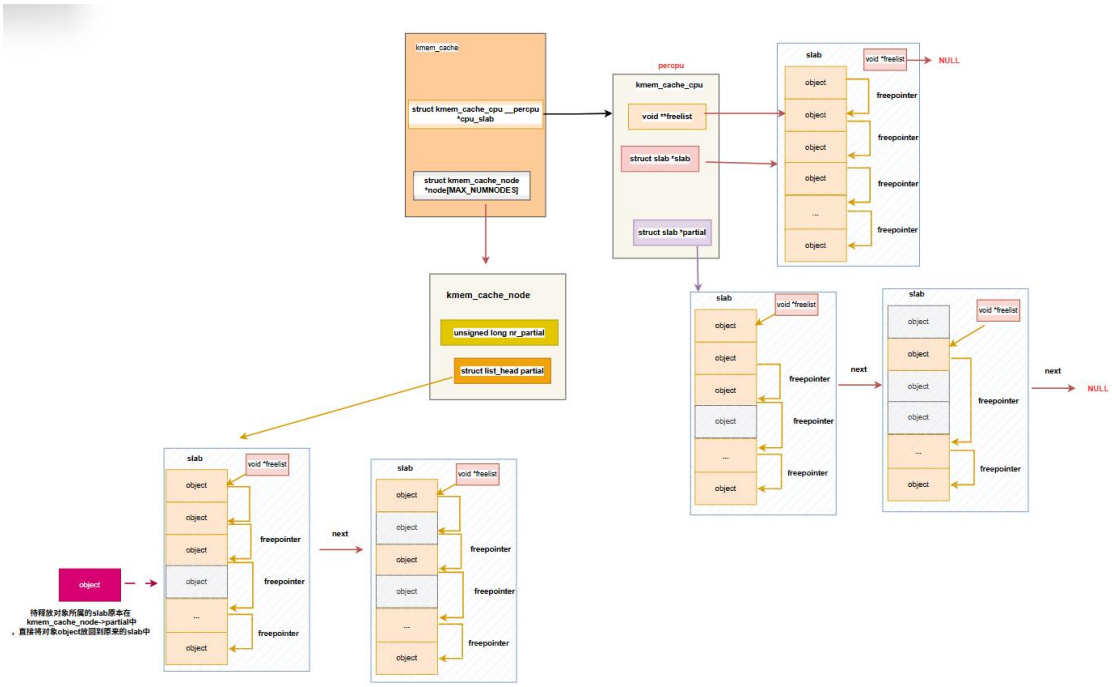
如果释放对象所属的 slab 原来是一个 partial slab，在对象释放之后变成了一个 empty slab，在这种情况下，内核将会把该 slab 插入到 slab cache 的备用仓库 NUMA 节点缓存中。

因为 slab 之所以会变成 empty slab，表明该 slab 并不是一个活跃的 slab，内核已经好久没有从该 slab 中分配对象了，所以只能把它释放回 kmem_cache_node->partial 链表中作为本地 cpu 缓存的后备选项。

但是 kmem_cache_node->partial 链表中的 slab 不可能是无限增长的，链表中缓存的 slab 个数受到 kmem_cache 结构中 **min_partial** 属性的限制：

```
struct kmem_cache {
        ......
        //主要控制 NUMA  节点缓存 partial 列表 slab 个数，
        //如果超过该值，那么列表中空闲的 empty slab 就会被释放回伙伴系统中。
        unsigned long min_partial;
        ......


}
```

所以内核在将 slab 插入到 kmem_cache_node->partial 链表之前，需要检查当前 kmem_cache_node->partial 链表中缓存的 slab 个数 nr_partial 是否已经超过了 min_partial 的限制。如果超过了限制，则直接将 slab 释放回伙伴系统中，如果没有超过限制，才会将 slab 插入到 kmem_cache_node->partial 链表中。



若释放对象所属的 slab 本来就在 kmem_cache_node->partial 链表中，这种情况下就是直接释放对象回 kmem_cache_node->partial 中的 slab，不需要迁移 slab 的位置。